Powiadomienia i Web Push

Na tych zajęciach poznamy trzy sposoby wyświetlania powiadomień w aplikacji PWA:

1. **Notification API** — powiadomienie tworzone przez skrypt strony (`new Notification(...)`), działa tylko wtedy, gdy strona jest otwarta,
2. **powiadomienie z service workera** (`registration.showNotification(...)`) — działa także na Androidzie i pozwala obsłużyć kliknięcie w powiadomienie,
3. **Web Push (Push API)** — wiadomość wysłana **z serwera**, która dociera do przeglądarki nawet wtedy, gdy karta z aplikacją jest zamknięta. Przeglądarka budzi wtedy service workera, a ten wyświetla powiadomienie.

Obsługa w przeglądarkach:

- Chrome, Edge, Firefox (komputer i Android) — obsługują wszystkie trzy sposoby, z jednym wyjątkiem: na Androidzie **nie działa** `new Notification(...)` (tylko powiadomienia z service workera),
- Safari na macOS — obsługuje wszystkie trzy sposoby,
- iPhone/iPad (iOS 16.4+) — powiadomienia i Web Push działają **tylko w aplikacji dodanej do ekranu początkowego** (Safari → **Udostępnij** → **Do ekranu początkowego**), a nie w zwykłej karcie Safari.

Przygotowanie projektu

W pracowni Node.js jest już zainstalowany — sprawdzamy tylko jego wersję (potrzebna jest wersja LTS, tak jak w lab01):

```
node --version
```

Praca w domu: jeśli polecenie nie działa albo wersja jest starsza, instalujemy aktualną wersję LTS ze strony https://nodejs.org/en/download/ (po instalacji trzeba otworzyć terminal na nowo).

Tworzymy folder `lab_02`, a w nim podfolder `public` — tam będzie nasza aplikacja. Do folderu `public` kopiujemy z lab01:

- grafikę `android.png` (użyjemy jej jako ikony powiadomień),
- plik `manifest.json` oraz folder `icons` z ikonami aplikacji (manifest jest potrzebny, aby aplikację dało się zainstalować — na iPhonie bez instalacji powiadomienia w ogóle nie działają). W manifeście zmieniamy `name` i `short_name`, np. na `Powiadomienia` i `Powiad.`.

W folderze `public` tworzymy jeszcze trzy pliki: `index.html`, `notification.js` i `service-worker.js`.

W pliku `index.html` umieszczamy:

```
<!DOCTYPE html>
<html lang="pl">
  <head>
    <meta charset="UTF-8" />
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="theme-color" content="#000000" />
    <title>Powiadomienia</title>
    <link rel="manifest" href="manifest.json" />
    <script src="notification.js" defer></script>
  </head>
  <body>
    <h1>Notification API</h1>
    <p>Uprawnienie do powiadomień: <strong id="permission">?</strong></p>
    <button id="enable">Włącz powiadomienia</button>
    <button id="local">Powiadomienie ze strony</button>
    <button id="sw">Powiadomienie z service workera</button>
  </body>
</html>
```

Plik `service-worker.js` na razie zostawiamy pusty, a stronę uruchamiamy przez lokalny serwer — w folderze `public` wpisujemy:

```
npx serve
```

i otwieramy w Chrome podany adres (np. `http://localhost:3000`). **Nie otwieramy** pliku `index.html` dwuklikiem — na adresie `file://...` powiadomienia ani service worker nie działają. Nie potrzebujemy za to HTTPS: adres `localhost` przeglądarka traktuje jako bezpieczny kontekst.

Prośba o uprawnienia

Zanim wyświetlimy jakiekolwiek powiadomienie, użytkownik musi się na nie zgodzić. Uprawnienie (`Notification.permission`) może mieć trzy wartości:

- `default` — użytkownik jeszcze nie odpowiedział,
- `granted` — zgoda,
- `denied` — odmowa (strona nie może już ponownie zapytać — zmienić to może tylko użytkownik w ustawieniach przeglądarki).

O zgodę pytamy **dopiero po kliknięciu przycisku**, a nie od razu po wejściu na stronę. Firefox i Safari w ogóle nie pokażą prośby, która nie wynika z działania użytkownika, a Chrome prośby wyświetlane zaraz po załadowaniu strony często zamienia na mało widoczną ikonę w pasku adresu. Do pliku `notification.js` wpisujemy:

```
const permissionInfo = document.querySelector('#permission');

function updatePermissionInfo() {
  permissionInfo.textContent =
    'Notification' in window ? Notification.permission : 'brak obsługi w tej przeglądarce';
}

updatePermissionInfo();

document.querySelector('#enable').addEventListener('click', async () => {
  const permission = await Notification.requestPermission();
```

```
  console.log('Uprawnienie:', permission);
  updatePermissionInfo();
});
```

Po kliknięciu **Włącz powiadomienia** pod paskiem adresu pojawi się pytanie, czy strona może wyświetlać powiadomienia — wybieramy **Zezwól**. Na stronie powinna pojawić się wartość `granted`.

Jeśli wcześniej odmówiliśmy (albo kilka razy zamknęliśmy pytanie bez odpowiedzi — Chrome sam ustawia wtedy blokadę), uprawnienie ma wartość `denied`. Aby to zmienić, klikamy ikonę **ustawień strony** po lewej stronie adresu (w nowszym Chrome zamiast kłódki są tam dwa suwaki), włączamy **Powiadomienia** albo wybieramy **Zresetuj uprawnienia**, a następnie odświeżamy stronę.

Uwaga (Windows): jeśli mamy zgodę (`granted`), a powiadomienia się nie pojawiają, sprawdzamy w **Ustawieniach systemu** → **System** → **Powiadomienia**, czy powiadomienia są włączone dla Chrome oraz czy nie jest włączony tryb **Nie przeszkadzać**.

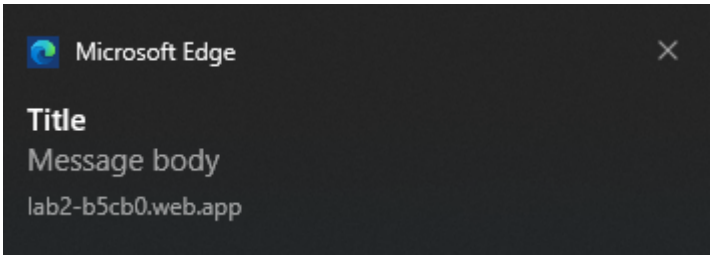
Pierwsze powiadomienie

Na końcu pliku `notification.js` dopisujemy funkcję, która tworzy powiadomienie o tytule `Title` i treści `Message body`, oraz podpinamy ją pod drugi przycisk:

```
function showLocalNotification() {
  if (Notification.permission !== 'granted') {
    alert('Najpierw włącz powiadomienia.');
    return;
  }
  try {
    const notification = new Notification('Title', {
      body: 'Message body',
    });
    notification.onclick = () => {
      window.focus();
      notification.close();
    };
  } catch (err) {
    // np. Chrome na Androidzie: "Illegal constructor" - tam działa tylko powiadomienie z
service workera
    alert('Ta przeglądarka nie obsługuje new Notification(): ' + err.message);
  }
}

document.querySelector('#local').addEventListener('click', showLocalNotification);
```

Po kliknięciu **Powiadomienie ze strony** powinno pojawić się powiadomienie systemowe (na zrzucie — Microsoft Edge w Windows):

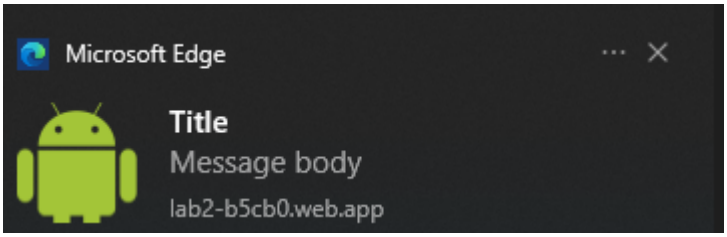


Kliknięcie w powiadomienie (`onclick`) przenosi nas z powrotem do okna ze stroną i zamyka powiadomienie.

Teraz zmieńmy ikonę powiadomienia. Do opcji powiadomienia dodajemy właściwość `icon` ze **ścieżką względną** do obrazka z folderu `public`:

```
const notification = new Notification('Title', {
  body: 'Message body',
  icon: 'android.png',
});
```

Po odświeżeniu strony i ponownym kliknięciu przycisku powiadomienie powinno mieć ikonę:



Inne przydatne opcje powiadomienia (pełna lista: Notification() w MDN):

- `tag` — identyfikator; nowe powiadomienie z tym samym `tag` zastępuje poprzednie zamiast dokładać kolejne,
- `requireInteraction: true` — powiadomienie nie znika samo, dopóki użytkownik go nie kliknie lub nie zamknie (obsługiwane m.in. w Chrome na komputerze),
- `silent: true` — bez dźwięku i wibracji,
- `data` — dowolne dane dołączone do powiadomienia (skorzystamy z tego w service workerze).

Ograniczenia `new Notification(...)`: działa tylko wtedy, gdy strona jest otwarta, a w Chrome na Androidzie w ogóle nie jest dostępne (zgłasza błąd `Illegal constructor`). Dlatego w aplikacjach PWA powiadomienia wyświetlamy zwykle przez service workera.

Powiadomienie z service workera

Do pliku `service-worker.js` wklejamy:

```
// Nowa wersja service workera od razu przejmuje stronę (bez czekania na zamknięcie kart).
self.addEventListener('install', () => self.skipWaiting());
self.addEventListener('activate', (event) => event.waitUntil(self.clients.claim()));
```

```
// Kliknięcie w powiadomienie: zamykamy je i otwieramy adres zapisany w "data.url".
self.addEventListener('notificationclick', (event) => {
  event.notification.close();
  const url = event.notification.data?.url || './';
  event.waitUntil(self.clients.openWindow(url));
});
```

W tym labie service worker **niczego nie zapisuje w cache** — dzięki temu każda zmiana plików jest od razu widoczna po odświeżeniu strony. Dzięki `skipWaiting()` i `clients.claim()` po zmianie `service-worker.js` wystarczy odświeżyć stronę (nie trzeba zamykać kart, jak w lab01).

Jeśli mimo to zmiany w `service-worker.js` nie działają, sprawdzamy w DevTools (**F12**) → **Application** → **Service workers** godzinę przy **Received** — to moment pobrania service workera przez przeglądarkę. Jeśli jest wcześniejsza niż ostatni zapis pliku, klikamy **Update** (a gdy to nie pomoże — **Unregister** i odświeżamy stronę).

Na końcu pliku `notification.js` rejestrujemy service workera i dopisujemy funkcję dla trzeciego przycisku:

```
if ('serviceWorker' in navigator) {
  navigator.serviceWorker
    .register('./service-worker.js')
    .catch((err) => console.error('Błąd rejestracji service workera:', err));
}

async function showSwNotification() {
  if (Notification.permission !== 'granted') {
    alert('Najpierw włącz powiadomienia.');
    return;
  }
  const registration = await navigator.serviceWorker.ready;
  await registration.showNotification('Powiadomienie z service workera', {
    body: 'Kliknij, aby otworzyć stronę uczelni.',
    icon: 'android.png',
    data: { url: 'https://www.uws.edu.pl' },
  });
}

document.querySelector('#sw').addEventListener('click', showSwNotification);
```

Ścieżka `./service-worker.js` jest **względna** (tak jak w lab01) — dzięki temu aplikacja zadziała także w podkatalogu, np. na GitHub Pages. `navigator.serviceWorker.ready` czeka, aż service worker będzie aktywny, i zwraca jego rejestrację — metoda `showNotification()` przyjmuje te same opcje co `new Notification()`.

Odświeżamy stronę i klikamy **Powiadomienie z service workera**. Po kliknięciu w powiadomienie powinna otworzyć się strona uczelni — tę akcję wykonuje service worker (zdarzenie `notificationclick`), więc zadziała nawet wtedy, gdy strona z aplikacją jest już zamknięta. W DevTools (**F12**) → **Application** → **Service workers** service worker powinien mieć status **activated and is running**.

Web Push

Do tej pory powiadomienie tworzyła sama strona. **Web Push** pozwala wysłać powiadomienie z serwera — np. o nowej wiadomości w komunikatorze — także wtedy, gdy użytkownik nie ma otwartej naszej strony. Biorą w tym udział trzy strony:

1. **przeglądarka** — zapisuje się na powiadomienia (`pushManager.subscribe()`) i dostaje **subskrypcję**: adres URL (`endpoint`) oraz klucze szyfrujące,
2. **usługa push** przeglądarki — serwer producenta przeglądarki, który dostarcza wiadomości do urządzenia (Chrome i Edge korzystają z serwerów Google/Microsoft, Firefox z Mozilli, Safari z Apple). Nie zakładamy tam żadnego konta,
3. **nasz serwer** — wysyła zaszyfrowaną wiadomość na adres `endpoint`. Podpisuje ją kluczem prywatnym **VAPID**, dzięki czemu usługa push wie, że wiadomość pochodzi od właściciela aplikacji. Klucz publiczny VAPID przekazujemy przeglądarce przy subskrypcji.

Gdy wiadomość dotrze, przeglądarka budzi naszego service workera zdarzeniem `push`, a ten **musi** wyświetlić powiadomienie (Chrome wymaga tego opcją `userVisibleOnly: true`).

Rolę „serwera" będzie u nas pełnił skrypt Node.js uruchamiany z terminala, korzystający z biblioteki web-push.

Klucze VAPID

Od tej pory pracujemy w dwóch folderach. Pliki strony zostają w `public`, a skrypty Node.js i klucze tworzymy **piętro wyżej**, w folderze `lab_02` — w `public` trafiają tylko pliki publikowane w Internecie. Po wykonaniu części „Web Push" folder będzie wyglądał tak (publikacja na Firebase doda jeszcze m.in. `firebase.json` i `.firebaserc`):

```
lab_02/
├── public/                ← strona (publikowana w Internecie)
│   ├── icons/
│   ├── android.png
│   ├── index.html
│   ├── manifest.json
│   ├── notification.js
│   └── service-worker.js
├── node_modules/
├── .gitignore
├── generate-keys.mjs      ← skrypty i klucze (NIE w public!)
├── package.json
├── package-lock.json
├── send-push.mjs
├── subscription.json
└── vapid-keys.json
```

W pierwszym terminalu nadal działa `npx serve` (w folderze `public`) — nie zatrzymujemy go. Otwieramy **drugi terminal** i przechodzimy w nim do folderu `lab_02` (z `public` poleceniem `cd ..`). Wszystkie polecenia `npm` i `node` z tej części wpisujemy w tym drugim terminalu. W folderze `lab_02` wpisujemy:

```
npm init -y
npm install web-push
```

Tworzymy w folderze `lab_02` plik `generate-keys.mjs`:

```
import { existsSync, readFileSync, writeFileSync } from 'node:fs';
import webpush from 'web-push';

// Klucze generujemy tylko raz - nowe klucze unieważniają wszystkie dotychczasowe
subskrypcje.
if (!existsSync('vapid-keys.json')) {
  writeFileSync('vapid-keys.json', JSON.stringify(webpush.generateVAPIDKeys(), null, 2));
}
const keys = JSON.parse(readFileSync('vapid-keys.json', 'utf8'));
console.log('Klucz publiczny VAPID (wklej do public/notification.js):');
console.log(keys.publicKey);
```

i uruchamiamy go:

```
node generate-keys.mjs
```

Powstanie plik `vapid-keys.json` z kluczem publicznym (`publicKey`) i prywatnym (`privateKey`), a w terminalu pojawi się klucz publiczny — kopiujemy go.

> ⚠️ **Klucz prywatny jest tajny.** Kto go zna, może wysyłać powiadomienia w imieniu naszej aplikacji. Plik `vapid-keys.json` **nie może** trafić do repozytorium ani do folderu `public`. W folderze `lab_02` tworzymy (lub uzupełniamy) plik `.gitignore`:
>
> ```
> node_modules/
> vapid-keys.json
> subscription.json
> ```

Subskrypcja w przeglądarce

W pliku `index.html` przed `</body>` dopisujemy:

```
    <h2>Web Push</h2>
    <button id="subscribe">Subskrybuj powiadomienia push</button>
    <button id="unsubscribe">Anuluj subskrypcję</button>
    <p>
      <label for="subscription">Subskrypcja (skopiuj do pliku lab_02/subscription.json):
</label><br />
      <textarea id="subscription" rows="10" cols="60" readonly></textarea>
    </p>
    <button id="copy">Kopiuj subskrypcję</button>
```

Na końcu pliku `notification.js` dopisujemy (w miejsce `TU_WKLEJ_KLUCZ_PUBLICZNY` wklejamy klucz publiczny z terminala):

```
const VAPID_PUBLIC_KEY = 'TU_WKLEJ_KLUCZ_PUBLICZNY';
const subscriptionField = document.querySelector('#subscription');

// Klucz VAPID jest zapisany w base64url, a pushManager.subscribe() oczekuje tablicy bajtów.
function urlBase64ToUint8Array(base64String) {
  const padding = '='.repeat((4 - (base64String.length % 4)) % 4);
  const base64 = (base64String + padding).replace(/-/g, '+').replace(/_/g, '/');
  return Uint8Array.from(atob(base64), (c) => c.charCodeAt(0));
}

async function subscribePush() {
  if (!('PushManager' in window)) {
    alert('Ta przeglądarka nie obsługuje Web Push (na iPhonie: najpierw dodaj aplikację do
ekranu początkowego).');
    return;
  }
  const registration = await navigator.serviceWorker.ready;
  let subscription = await registration.pushManager.getSubscription();
  if (!subscription) {
    try {
      // Jeśli użytkownik jeszcze nie wyraził zgody, przeglądarka zapyta o nią teraz.
      subscription = await registration.pushManager.subscribe({
        userVisibleOnly: true,
        applicationServerKey: urlBase64ToUint8Array(VAPID_PUBLIC_KEY),
      });
    } catch (err) {
      alert('Nie udało się zasubskrybować: ' + err.message);
      return;
    } finally {
      updatePermissionInfo();
    }
  }
  subscriptionField.value = JSON.stringify(subscription, null, 2);
}

async function unsubscribePush() {
  const registration = await navigator.serviceWorker.ready;
  const subscription = await registration.pushManager.getSubscription();
  if (subscription) {
    await subscription.unsubscribe();
  }
  subscriptionField.value = '';
}

document.querySelector('#subscribe').addEventListener('click', subscribePush);
document.querySelector('#unsubscribe').addEventListener('click', unsubscribePush);
document.querySelector('#copy').addEventListener('click', async () => {
  try {
    await navigator.clipboard.writeText(subscriptionField.value);
    alert('Skopiowano subskrypcję do schowka.');
  } catch (err) {
    // brak dostępu do schowka - zaznaczamy tekst, żeby skopiować go ręcznie (Ctrl+C)
```

```
    subscriptionField.select();
    alert('Nie udało się skopiować automatycznie - tekst jest zaznaczony, skopiuj go
ręcznie.');
  }
});
```

Do pliku `service-worker.js` dopisujemy obsługę zdarzenia `push`:

```
// Wiadomość z serwera: oczekujemy JSON-a {"title": "...", "body": "..."},
// ale obsługujemy też zwykły tekst (np. z przycisku "Push" w DevTools).
self.addEventListener('push', (event) => {
  let message = { title: 'Web Push', body: event.data ? event.data.text() : '' };
  try {
    message = event.data.json();
  } catch (err) {
    // to nie był JSON - zostaje tekst
  }
  event.waitUntil(
    self.registration.showNotification(message.title, {
      body: message.body,
      icon: 'android.png',
      data: { url: './' },
    })
  );
});
```

`event.waitUntil()` informuje przeglądarkę, że service worker ma jeszcze pracę — bez tego mógłby zostać uśpiony, zanim wyświetli powiadomienie.

Odświeżamy stronę. Najpierw sprawdzamy sam service worker, bez serwera: w DevTools → **Application** → **Service workers** przy naszym service workerze jest pole **Push** z przykładowym tekstem — klikamy przycisk **Push** obok niego. Powinno pojawić się powiadomienie *Web Push* z tym tekstem. Jeśli nic się nie dzieje (i nie ma błędów), przeglądarka najpewniej używa jeszcze starej wersji service workera — bez obsługi zdarzenia `push`. Sprawdzamy godzinę przy **Received** i klikamy **Update** (patrz „Powiadomienie z service workera").

Następnie klikamy **Subskrybuj powiadomienia push**. W polu tekstowym pojawi się subskrypcja, np.:

```
{
  "endpoint": "https://fcm.googleapis.com/fcm/send/...",
  "expirationTime": null,
  "keys": {
    "p256dh": "...",
    "auth": "..."
  }
}
```

Adres `endpoint` zależy od przeglądarki (Chrome — `fcm.googleapis.com`, Edge — `notify.windows.com`, Firefox — `push.services.mozilla.com`). Klikamy **Kopiuj subskrypcję**, tworzymy w folderze `lab_02` (**nie** w

`public`) plik `subscription.json` i wklejamy do niego skopiowany tekst.

W prawdziwej aplikacji strona wysłałaby subskrypcję na serwer (np. `fetch()` metodą `POST`), a serwer zapisałby ją w bazie danych — po jednej dla każdego użytkownika i urządzenia. My kopiujemy ją ręcznie.

Wysłanie powiadomienia z serwera

Tworzymy w folderze `lab_02` (**nie** w `public`) plik `send-push.mjs` (w `mailto:` wpisujemy swój adres e-mail — usługa push może się z nami skontaktować, gdyby nasza aplikacja sprawiała problemy):

```
import { readFileSync } from 'node:fs';
import webpush from 'web-push';

// Odczyt pliku JSON (usuwamy ewentualny znacznik BOM, który dodają niektóre edytory w
Windows).
const readJson = (file) => JSON.parse(readFileSync(file, 'utf8').replace(/^/, ''));

const keys = readJson('vapid-keys.json');
const subscription = readJson('subscription.json');

webpush.setVapidDetails('mailto:twoj.adres@example.com', keys.publicKey, keys.privateKey);

const [title = 'Web Push', body = 'Wiadomość wysłana z serwera'] = process.argv.slice(2);

try {
  const result = await webpush.sendNotification(subscription, JSON.stringify({ title, body
}));
  console.log('Wysłano, status:', result.statusCode);
} catch (err) {
  console.error('Błąd wysyłania:', err.statusCode, err.body || err.message);
}
```

Wysyłamy powiadomienie (tytuł i treść podajemy w cudzysłowach):

```
node send-push.mjs "Nowa wiadomość" "Pozdrowienia z serwera!"
```

W terminalu powinien pojawić się komunikat **Wysłano, status: 201**, a po chwili — powiadomienie. Teraz najważniejszy test: **zamykamy kartę** z naszą aplikacją (przeglądarka musi pozostać uruchomiona) i ponownie wysyłamy powiadomienie. Powinno się pojawić, a kliknięcie w nie otworzy naszą aplikację.

Najczęstsze błędy przy wysyłaniu:

- **403** — subskrypcja została utworzona z innym kluczem publicznym niż ten w `vapid-keys.json` (np. po wygenerowaniu nowych kluczy). Klikamy **Anuluj subskrypcję**, sprawdzamy klucz w `notification.js`, subskrybujemy ponownie i zapisujemy nowy `subscription.json`,
- **404** lub **410** — subskrypcja wygasła lub została anulowana (np. po **Clear site data**) — subskrybujemy ponownie,
- `Unexpected token ... in JSON` — plik `subscription.json` nie zawiera poprawnego JSON-a (np. wkleiliśmy tylko fragment),

- `ENOENT: no such file or directory, open '...vapid-keys.json'` (albo `subscription.json`) — polecenie uruchomiliśmy w złym folderze albo plik leży w `public`. Skrypty, `vapid-keys.json` i `subscription.json` muszą być w `lab_02`, a polecenie `node send-push.mjs ...` wpisujemy w folderze `lab_02` (patrz schemat w „Klucze VAPID").

Jeśli status to **201**, a powiadomienie się nie pojawia, sprawdzamy ustawienia powiadomień w Windows (patrz „Prośba o uprawnienia") oraz czy w DevTools → **Application** → **Service workers** nie ma błędów.

Test na telefonie

Na telefonie aplikacja musi być dostępna przez HTTPS, więc publikujemy ją na Firebase Hosting — dokładnie tak jak w lab01 (sekcja „Publikacja z szyfrowanym połączeniem (HTTPS)"), z dwiema różnicami:

- w konsoli Firebase tworzymy **nowy projekt** (np. `lab02-nazwisko`) — gdybyśmy wybrali projekt z lab01, `firebase deploy` zastąpiłby opublikowaną tam aplikację,
- `firebase init hosting` uruchamiamy w folderze `lab_02`, jako folder publiczny podajemy `public`, a na pytanie **File public/index.html already exists. Overwrite?** odpowiadamy `N`.

Publikowany jest tylko folder `public`, więc klucz prywatny z `vapid-keys.json` nie trafi do Internetu.

> ⚠️ **Koniec zajęć w pracowni:** wylogowujemy się z Firebase CLI poleceniem `firebase logout`.

Na telefonie z Androidem otwieramy w Chrome adres **Hosting URL**, klikamy **Włącz powiadomienia** i sprawdzamy oba rodzaje powiadomień — przycisk **Powiadomienie ze strony** powinien pokazać komunikat o braku obsługi `new Notification()`, a **Powiadomienie z service workera** powinno zadziałać.

Następnie na telefonie klikamy **Subskrybuj powiadomienia push** i **Kopiuj subskrypcję**, a tekst przesyłamy na komputer (np. e-mailem do siebie) i zapisujemy jako `subscription.json`. Zamykamy Chrome na telefonie i wysyłamy powiadomienie poleceniem `node send-push.mjs ...` — powinno pojawić się na telefonie.

Na iPhonie (iOS 16.4 lub nowszy) najpierw dodajemy aplikację do ekranu początkowego (Safari → **Udostępnij** → **Do ekranu początkowego**), uruchamiamy ją z ikony i dopiero tam włączamy powiadomienia i subskrypcję.

Zadanie

Na podstawie powyższej instrukcji przygotuj aplikację PWA z powiadomieniami, która spełnia poniższe warunki:

1. O zgodę na powiadomienia aplikacja pyta dopiero po kliknięciu przycisku, a aktualny stan uprawnienia (`default` / `granted` / `denied`) jest widoczny na stronie.
2. Aplikacja zawiera **formularz** z polami: tytuł, treść oraz ikona (wybór jednego z co najmniej trzech obrazków z folderu `public`, np. listą `<select>`). Po wysłaniu formularza wyświetla się powiadomienie z podanymi danymi — przez service workera, tak aby działało także na Androidzie. Puste pole tytułu nie powinno wyświetlać powiadomienia (walidacja formularza).
3. Kliknięcie w powiadomienie otwiera (lub przywołuje) okno aplikacji.
4. Aplikacja obsługuje **Web Push**: subskrypcję, anulowanie subskrypcji i wyświetlenie powiadomienia wysłanego skryptem `send-push.mjs` — także wtedy, gdy karta z aplikacją jest zamknięta.

5. Aplikacja jest opublikowana przez HTTPS (Firebase Hosting lub GitHub Pages), a powiadomienie push dociera na telefon.
6. Klucz prywatny VAPID (`vapid-keys.json`) **nie znajduje się** w repozytorium.

Kod (folder `public` oraz skrypty `generate-keys.mjs`, `send-push.mjs`, `package.json` i `.gitignore`) umieść w repozytorium GitHub w folderze `lab_02/`. W pliku `README.md` podaj adres opublikowanej aplikacji oraz zrzut ekranu powiadomienia push na telefonie.

Źródła:

- https://developer.mozilla.org/en-US/docs/Web/API/Notifications_API/Using_the_Notifications_API
- https://developer.mozilla.org/en-US/docs/Web/API/Notification
- https://developer.mozilla.org/en-US/docs/Web/API/ServiceWorkerRegistration/showNotification
- https://developer.mozilla.org/en-US/docs/Web/API/Push_API
- https://web.dev/articles/push-notifications-overview
- https://web.dev/articles/push-notifications-web-push-protocol
- https://github.com/web-push-libs/web-push
- https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/

Przed następnymi zajęciami: konto OpenWeatherMap

Na następnych zajęciach zbudujemy aplikację pogodową, która pobiera dane z serwisu **OpenWeatherMap**. Serwis wymaga **klucza API** (identyfikatora aplikacji), a nowy klucz zaczyna działać dopiero **po pewnym czasie** (zwykle kilkadziesiąt minut, czasem do kilku godzin) — dlatego konto zakładamy **już teraz**:

1. Zakładamy darmowe konto na https://home.openweathermap.org/users/sign_up i potwierdzamy adres e-mail.
2. Klucz znajdziemy w zakładce **API keys** (https://home.openweathermap.org/api_keys) — na następnych zajęciach wkleimy go do kodu aplikacji.

Korzystamy z usługi **Current Weather Data** (bieżąca pogoda), która jest dostępna w darmowym planie — domyślny klucz utworzony przy rejestracji wystarcza. **Nie wybieramy żadnego planu ani subskrypcji i nie podajemy karty płatniczej.** Usługa **One Call API 3.0**, mocno reklamowana na stronie z cennikiem, wymaga osobnej subskrypcji z kartą — nie jest nam potrzebna.